HexaFuture

HexaShell: A Thin Harness for Exploring the Limits of VLM Agents as Robot Coding Agents

Hongcheng Gao*† Alex Zhou* Jay Zhu Zelin Zheng Shijia Ge Xiping Li
Di Wu Yazhe Wang Jianshu Zeng Xuan Shangguan Yongli Wang
Yexing Wan Lingyu He Zhiqi Jia Sihang Wu Xiao He†

*Equal Contribution †Corresponding Author

hexafuture.ai

Abstract

Agents built on vision language models (VLMs) now compete with learned policies on robot manipulation leaderboards. We ask how far a coding agent can go when it controls a robot instead of a computer, and where it stops. Our system, HexaShell, gives the agent a thin harness: one command line that turns its commands into robot motion and makes no decisions of its own. On the robot, the agent keeps doing what it does best: it reads output, writes code and runs tests. Two agents of the same model share the work. An acting agent runs each episode. After a failure, an optimizer agent reads the full record of the episode, with every command, image and message, finds the cause, and writes or fixes small programs called tools, each with its own tests. What the agents learn stays as code, and no demonstrations or training are needed. At run time, the tools use only what a real robot also has: RGB images, camera calibration and the state of the arms. They find objects by triangulation between camera views. Because these inputs are the same in simulation and on hardware, the tools move to a real robot with a different arm after only a small change for the arm. We build the tools on ten layouts of one seed per task with Codex and GPT-6-Astra and freeze them; the eight Open tasks share one set of tools. In RoboDojo's official evaluation on a hidden seed, 50 episodes for each of the 42 tasks, HexaShell reaches 46.9% success and a progress score of 53.2, the state of the art on the official RoboDojo leaderboard. The limit is contact, not reasoning: memory tasks reach 81.0%, but precision tasks only 32.0%. We open source the tools evolved so far at `https://github.com/HexaFuture/HexaShell`.

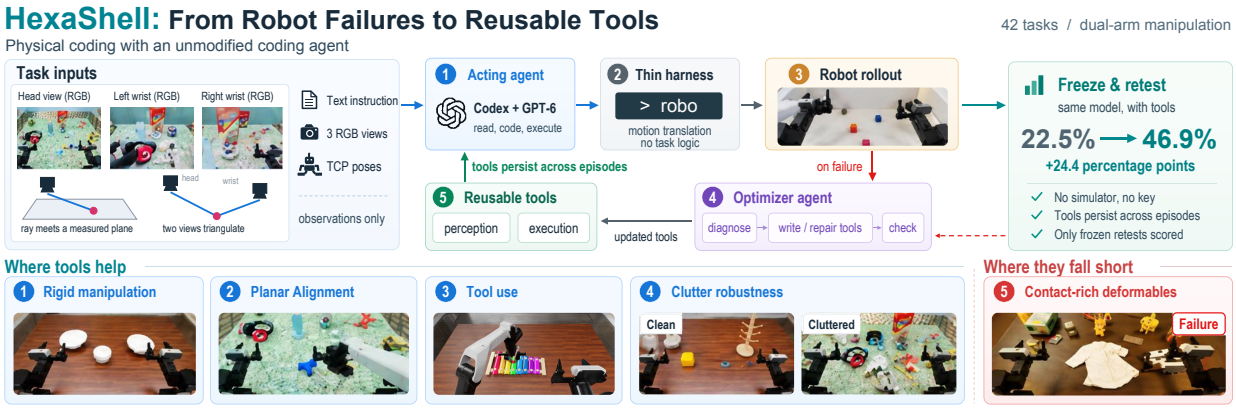


Figure 1. HexaShell turns robot failures into reusable tools. A coding agent operates a robot with two arms through a thin harness and sees RGB images only. After a failure, an optimizer agent diagnoses the episode and writes or fixes tools that use only what the robot observes. Both agents are Codex with GPT-6-Astra. Bottom: where tools help (rigid, planar, tool use, clutter) and where they fall short (cloth).



Figure 2. RoboDojo leaderboard. Altitude is the progress score on the leaderboard's metric. HexaShell reaches the state of the art in RoboDojo's official evaluation.

1 Introduction

Robot manipulation leaderboards have been led by learned policies. Vision language action models and world action models learn from large sets of demonstrations and output the robot's actions directly [4, 7, 35]. Agents built on vision language models have now started to compete. On RoboDojo [14], the model we use reaches 22.5% without tools of its own, choosing gripper targets through a fixed harness written by the benchmark's authors [69].

Coding agents suggest another route. In a software repository, a coding agent reads the code, runs it, reads the error and fixes the code, and the fix stays. Recent work has begun to let coding agents write and revise robot tools and programs [25, 26, 34, 40]. We ask how far a coding agent can go when it works on a robot in the same way, and where it stops. Our system, HexaShell, keeps the harness thin. The agent gets one command, robo, three RGB images with their calibration, and a manual of one page. The harness turns commands into joint motion and does nothing else. What the agent learns is kept as tools, small programs that add commands to robo and may use only what the robot observes. An acting agent tries a layout with the current tools. After a failure, an optimizer agent of the same model reads the episode as it would read a failing test, writes or fixes a tool with its unit tests, and the layout is tried again (Fig. 4). Both agents are Codex with GPT-6-Astra.

The hard part is turning pixels into positions. In a repository, everything the agent works on is text it can read exactly. On a robot, the agent sees objects as pixels, but every command needs a position in the world, to about a centimetre for a grasp. Vision language models are known to judge such distances poorly [9, 16]. The standard observation has no depth. A monocular depth network can estimate depth from one image, but on RoboDojo scenes Depth Anything V2 [65] is off by a median of 41 mm at the head camera and about 100 mm at the wrists, even after we fix its scale with the table. Our tools use triangulation instead. The cameras are calibrated, and the robot knows where its wrists are. A point that two cameras see lies where their rays meet, and a point on a measured plane, such as the table, lies where its ray meets that plane [27]. When one view is not enough, the robot moves a wrist camera to get another. This is active perception [3], and on a benchmark it has a price: the motion uses action steps, and every task has a step limit.

We build the tools on ten layouts of each task and freeze them. RoboDojo then evaluated the system in its official

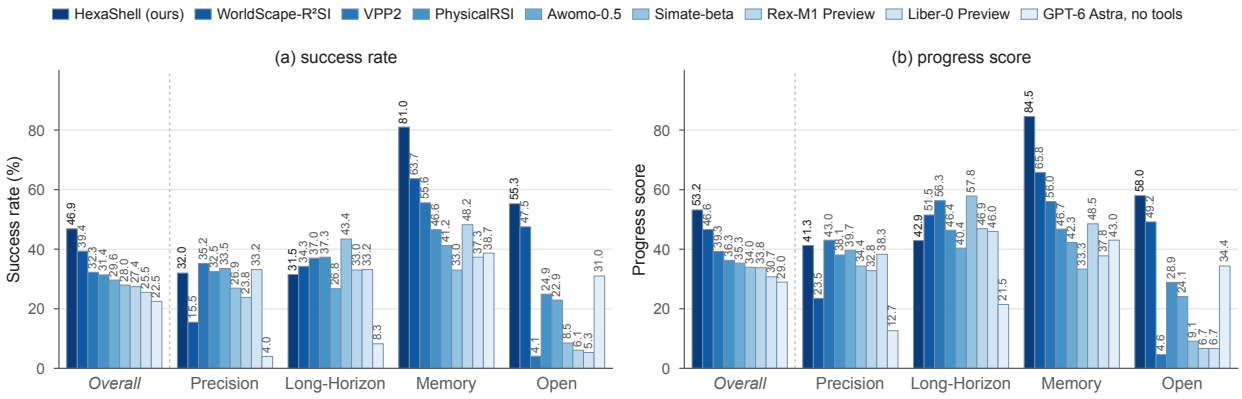


Figure 3. RoboDojo comparison on the leaderboard's metrics: success rate (a) and progress score (b). Overall is the leaderboard's score, the mean of its five task types (Generalization: mean of its standard and cluttered variants). HexaShell: RoboDojo's official evaluation on a hidden seed, 50 episodes per task, with RGB images only and frozen tools (Sec. 4.1). GPT-6-Astra is the same model without tools [69]. Systems are ordered and shaded by Overall success rate.

evaluation, on a hidden seed with 50 episodes per task. On all 42 tasks, HexaShell reaches 46.9% success and a progress score of 53.2 on the leaderboard's metric, the state of the art on the official RoboDojo leaderboard (Figs. 2 and 3). It does best where the right action follows from measured geometry, and it stops at contact: tight insertion stays low. Over 420 retest episodes, success follows the contact a task needs, not what the agent must read or remember, and how an episode fails separates a gap in perception from a gap in execution (Sec. 5). The paper makes three contributions.

- A thin harness that leaves every decision to the agent. The harness gives an unchanged coding agent one command line to the robot and only turns commands into joint motion, so on the robot the agent still reads output, writes code and runs tests. With it, the same model rises from 22.5% through the benchmark authors' fixed harness to 46.9% in RoboDojo's official evaluation, a new state of the art (Sec. 4.2).
- An acting agent and an optimizer agent that turn failures into tested tools. After a failure, the optimizer reads the full record of the episode, with every command, camera frame and message, finds the cause and edits one tool with its unit tests, and the same layout runs again. A round takes a median of 6.3 minutes, and no robot data, training or change to the model is needed (Sec. 3).
- Inputs limited to what any robot has, so the tools generalize. At run time, the tools see only RGB images, the camera calibration and the state of the arms, and measure positions by geometry. Tools built on ten layouts of one seed reach the state of the art on a hidden seed. With 13 to 18 extra objects they keep 82% of their success, more than any learned policy, and triangulation scores 8.7 points higher than a depth network on the same layouts (Tab. 1). A real robot has the same inputs: on two PiPER X arms, a different model from the simulated ARX X5, the perception and execution tools run after a small change for the arm (Sec. 4.7).

Seven case studies show the loop at work. Each follows a failed attempt through the optimizer's diagnosis and fix to the next attempt on the same layout (Sec. 6).

2 Related Work

Language models have written robot code before. Code as Policies [37] emits executable programs over perception and control primitives, ProgPrompt [54] and Inner Monologue [29] plan with a language model and feedback, VoxPoser [30] writes value maps for a planner, ReKep [31] and MOKA [39] let a vision model mark keypoints that code then optimises, RoboCodeX [45] and RoboScript [10] generate manipulation code from images and text, language models write reward functions for policy learning [44, 66], and SayCan [1] grounds language in a fixed skill library. In each, the program is produced per instruction and discarded, and people write the primitives. HexaShell inverts this division of labour: the agent writes the primitives, they persist across episodes and are what is optimized, and the per-episode program is short. Concurrent work moves the same way: URAI pairs a programming agent that writes motion tools with an

execution agent that calls them [26], Gao *et al*. [25] treat tools, traces and memory as code that a physical coding agent revises, CaP-X benchmarks coding agents on manipulation [23], RLE-Bench tests them as robot learning engineers [43], LIBERO-Agent tests general agents that act on the robot directly [18], and MEMENTO evolves code policies with a memory of past programs [56]. We measure, on one full benchmark, how far such tools reach and where they stop.

Frontier models have also been used as policies without tools of their own. Zhang *et al*. [69] evaluate GPT-6-Astra on RoboDojo through a fixed end-effector harness with no learned policy in the loop, and in a case study the model writes a piano controller that transfers across tempo but not key geometry. Show-Harness [15], Agent as Policy [32], VIA [28] and World Action Agent [70] drive robots through a vision language agent and a fixed interface, and a line of recent work builds harnesses in which coding agents program, test and improve robot skills [11, 19, 21, 42, 57, 59, 63], sometimes starting from a few human demonstrations [34, 40]. We keep the model and benchmark and change one thing, letting the agent write, keep and repair tools (Sec. 5.1).

Agents that build their own tools are the closest family. Coding agents that read a repository, run the tests and fix the code [12, 33, 62, 64] are the starting point of this paper. Voyager [58] grows a skill library of JavaScript functions while playing Minecraft, Toolformer [52] learns when to call existing tools, LATM [8] and CRAFT [67] have language models write and retrieve tools, and self-improving coding agents [41, 47, 68] modify their own scaffolding or programs and keep edits that improve a score. Our loop belongs to this family but holds the agent's scaffold, prompt and base interface fixed and changes only task tools. We are able to diagnose faults in one round, while ensuring that the tools stay runnable under the official client. Like Reflexion [53] and Eureka [44], the optimizer iterates on code from traces, but acceptance is a fresh execution of the same layout rather than self-assessment.

Our tools measure 3D positions from RGB images. Monocular depth networks predict depth from one image [5, 60, 61, 65], but their scale must be fixed from outside and their errors are several centimetres on our scenes (Sec. 4.4). Vision language models judge metric distances in an image poorly, and SpatialVLM and SpatialRGPT train them on spatial data to improve this [9, 16]. Multiple view geometry recovers a point from two calibrated views [27], and plane sweep tests candidate depths by warping one view into another [17]; both are standard in OpenCV [6]. Active perception moves the sensor to get a better view [2, 3]. Our tools combine these ideas and charge each extra view to the robot's action budget, which a benchmark with a step limit makes explicit (Sec. 4.5).

The baselines we compare against are mostly learned manipulation policies. Vision language action models [4, 7, 35, 48–50, 73], trained on demonstrations collected with low cost hardware [71], are evaluated on suites such as LIBERO [22, 38], RoboCasa [46], RoboTwin [13] and SIMPLER [36], or in autonomous real-world evaluation [72]. RoboDojo [14] adds a fixed action budget per task, a multi-seed evaluation, and paired clean and cluttered layouts, which let us separate geometric competence from appearance memorisation (Sec. 4.3).

3 HexaShell

HexaShell has three parts (Fig. 4): a harness that turns commands into motion (Sec. 3.1), tools that measure 3D positions from RGB images (Secs. 3.2 and 3.3), and an evolution loop that turns failures into tools (Sec. 3.4). Four principles shaped them. The harness translates and never decides. Ground truth is isolated by architecture: the container the acting agent runs in holds no simulator, task or scoring code. Success is judged only by the benchmark's own checker. The manual states interfaces only; strategy lives in tools.

3.1 The harness

The acting agent is a stock coding agent CLI running in a container with no direct network access, no API key and no host directories. It is given one executable, robo, and a directory obs/ that is refreshed on request. The directory holds three 640×480 RGB images (head and two wrist cameras), each camera's intrinsics and its pose in the world, the two tool centre point (TCP) poses and the joint angles. There is no depth image. The head camera is fixed; each wrist camera pose follows from the TCP pose the robot reports. The manual has about sixty lines. The base commands are move (translate a TCP along world axes, at most 0.20 m per axis), rotate (about the TCP, at most 90° per axis), point (choose a gripper orientation from a small set and a finger opening axis), gripper, home, wait, obs, status and done. Every command returns a JSON record with a planning verdict, the reached pose and the remaining budget. Tools appear as additional sub-commands, and their interface.md files are appended to the manual under a heading "Extra tools for this task".

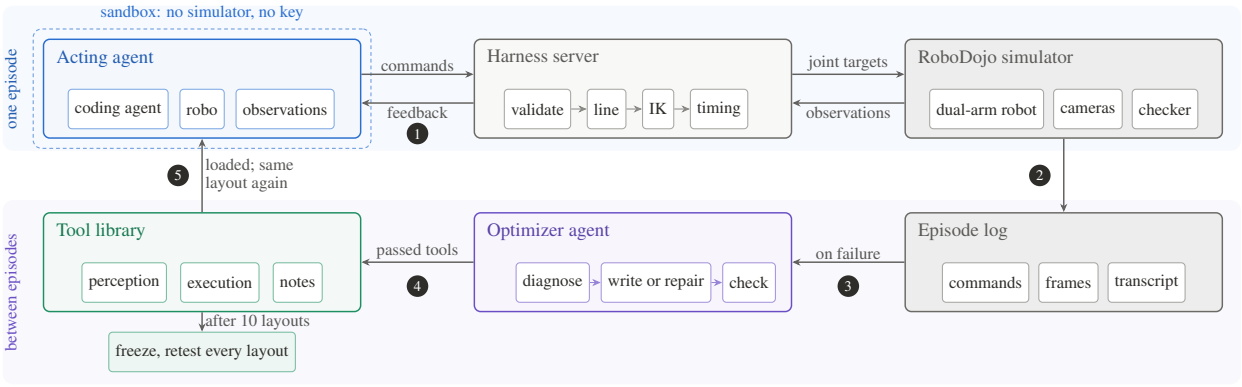


Figure 4. HexaShell. Top, one episode: ① a coding agent in a sandbox issues robo commands; the server validates each one, turns it into joint motion and returns feedback and RGB observations. Bottom, between episodes: ② the episode log keeps the commands, the camera frames and the agent's transcript; ③ after a failure the optimizer agent diagnoses the episode, ④ writes or fixes a tool that must pass a delivery check, and ⑤ the tool is loaded and the same layout is tried again.

Behind the command line, outbound traffic from the container passes through an egress proxy that adds the model credential and admits only the model endpoint and the harness's own routes. The server validates each command, interpolates a straight Cartesian path at 2 cm spacing, solves inverse kinematics per waypoint with cuRobo [55], and times the joint path with a trapezoidal profile under the robot's official limits. The timed path is sampled at the benchmark's 25 Hz action rate and sent through the official take_action call. There is no motion planner and no collision model beyond the robot and the table. RoboDojo limits each task by a count of action steps (step_lim, from 200 to 1900; 600 for push T, *i.e.* 24 s of robot time). Observations, measurements and model thinking cost nothing, and every executed step costs one. The counter is returned with every response.

The same server runs under two executors. A *direct* executor drives the RoboDojo environment in the same process and can reset, replay and log true object poses into a field the container never receives. A *bridge* executor wraps the same server as a policy for the official evaluation client, which has no reset and no ground truth. The agent's interface is the same under both, and every delivered tool must run under the bridge.

3.2 Measuring 3D from RGB images

A pixel and the calibration give a ray in the world frame. A ray alone is not a position, so every measurement adds one more constraint. The tools use three kinds, all plain OpenCV geometry [6, 27].

The first constraint is the table. A shared tool, table_plane, measures the table height once per episode, the first time it is called, at no step cost. It sweeps candidate heights, warps each camera image into another camera through the plane at that height, and keeps the height where the warped images agree best [17]. The three camera pairs agree to within 1 mm in our runs (0.7646–0.7656 m). The second constraint is any other plane whose height is known in the episode. The top face of a block, the rim of a bowl or a slot in a box lies on a horizontal plane; once its height is measured, the ray through a pixel on it gives the point. The third constraint is a second view. If two cameras see the same point, the two rays nearly meet, and the point between them is the triangulated position. Corresponding pixels come from the acting agent, who picks them in both images, or from template matching, contours and circle fits inside a tool.

The head camera sees the whole table, but often from too far away or at a poor angle, and objects hide each other. The wrist cameras then give the second view, but only after the robot moves them. A tool or the agent moves a wrist above the target, reads its pose from the TCP, and measures. These moves are robot motion, so they consume steps from the task's limit. This is the price of measuring without depth, and Sec. 4.5 measures it. The rule for both agents is that every number about an object must be measured in the episode. Object sizes, positions and heights may not come from the benchmark's assets, layouts, configuration or logs. A real robot offers the same inputs, RGB images, calibration and the state of the arms, so nothing the tools read at run time exists only in simulation (Sec. 4.7).

3.3 Tools

A tool is a directory with tool.py and interface.md. The Python file declares a name, its sub-commands and their arguments, and a run(api, command, args) function over an EpisodeAPI. This API offers joint angles, TCP poses, the three RGB images with calibration, and the same motion primitives the base commands use. Tools cannot import the simulator, cannot read object poses, and are loaded only if listed in the task's enabled_tools.txt. Measurement commands that move nothing are free, so the agent calls them freely. Commands that move an arm, including the ones that move a wrist camera to look, return the same planning verdict as base commands and list their stages. A shared pool holds two tools, pick and table_plane, which the loop never modifies. Everything else is task specific.

3.4 The evolution loop

One command starts a run for one task on one GPU. The loop uses the first ten layouts of evaluation seed 0 (for Generalization tasks, five standard and five cluttered, which share tools) and proceeds layout by layout. The acting agent attempts the layout with the current tools. On success, the optimizer agent reads the trajectory and adds a short entry to playbook.md on which tools worked and why. On failure, it reads the command log, the camera frames of every command and the current tool code. It writes a diagnosis to skill.md and adds or fixes a tool. A delivery check then verifies that files are complete, tools load, and the interface text names neither the task nor its objects; if the check fails the edit is reverted. The simulator is restarted and the *same* layout is attempted again, up to five edits per layout. After the tenth layout the optimizer consolidates its notes, the tool set is frozen and tested once more with no optimizer, on the ten development layouts and on ten new layouts (Sec. 4.1).

Both agents run GPT-6-Astra. The optimizer writes only under tasks/⟨task⟩/ and cannot touch the harness, the manual or the shared pool. The RGB tools come from one run per task with a development budget of 12 hours. Each run started from the tools of our earlier runs with depth, and for 25 tasks the optimizer also read a short note on what the checker requires, written from the public task definitions; the acting agent never sees the notes (Sec. D).

3.5 One setup for the Open tasks

The eight Open tasks are for evaluation only, so HexaShell runs them with one setup for all eight, built without any Open layout. The manual is the same for every Open task, and the prompt holds the instruction and the task's description and score table from the benchmark's documentation; there are no task tools and no notes. The tools come from the 34 other tasks. A perception tool, scene, separates objects from the table with a colour model of the table and masks out the arms from their joint angles. It then locates a chosen object from its outline in two views by a plane sweep and returns its top height, footprint and turn; if only one camera sees the object, it asks the agent to move a wrist camera. Eight shared skills, taken from the task tools of the other tasks, cover points and outlines on a plane, the rim of a container, guarded grasps and presses, bounded turns and timed picks on a moving belt. Two base commands are added: goto moves a TCP to an absolute pose and, if the pose cannot be reached, returns a nearby pose that can, and reach checks a pose without moving. The agent can also keep a Python session open across commands, with helper functions for rays, projections and triangulation, and the gripper reports whether its fingers stopped on an object and how wide they are open. We tested these tools only on scenes of the other 34 tasks, and then froze them.

4 Experiments

4.1 Setup

RoboDojo [14] has 42 simulated tasks for a dual-arm ARX X5 in Isaac Sim 5.1, grouped into Generalization (12 tasks, each with a cluttered variant that adds 13 to 18 distractor objects), Memory, Long-Horizon, Precision and Open. Each task ships three evaluation seeds; a seed is a set of 30 to 75 scene layouts in which object poses and table materials vary under fixed rules. Success is judged per step by the benchmark's own checker, and the episode ends at the first success. The leaderboard's score averages the five task types with equal weight, Generalization being the mean of its standard and cluttered variants; we also report the plain mean over the 54 variants.

We compare against the leaderboard as of 20 October 2026, where entries report 50 episodes on each of three seeds. The highest entries that output actions from a learned model are WorldScape-R^{2}SI (success rate 39.35%, progress score

46.58), a world action model directed by an agent, VPP2 (32.26%, 39.26), a world action model, and PhysicalRSI (31.38%, 36.27), a VLA directed by a language model agent. Thirty-one entries output actions from a learned policy, and we use all of them for the clutter comparison. The same model without tools is GPT-6-Astra choosing bounded gripper targets through the benchmark authors' fixed harness, 50 episodes on one seed per task (22.48%, 28.97) [69].

Our main system observes RGB images only and measures with the geometry of Sec. 3.2. Two variants use the same harness and model. In the first, a depth network, Depth Anything V2 Base [65], estimates depth inside the server for every image, and its scale is fixed in each frame by the table plane. In the second, the simulator renders depth, which the standard observation does not include. Both agents are the unmodified Codex command-line agent with GPT-6-Astra, with one RTX 4090 per run. A retest episode takes a median of 739 s for 26 s of robot time, almost all of it model latency.

The main result is RoboDojo's official evaluation of the frozen RGB system, which is on the official leaderboard. The benchmark's team ran it on its own system, on a hidden seed, with 50 episodes per task (25 standard and 25 cluttered for Generalization tasks). On the 34 tasks outside Open, the agent uses its frozen task tools. Its prompt also holds the task's description and score table from the benchmark's documentation and short notes on how the tools behaved in the development runs; the manual has revised rules for ending an episode. The eight Open tasks use one setup for all of them: the same manual and a shared set of tools, with no tools or notes for any single task (Sec. 3.5). The main results, Fig. 13 and Tab. 5 use this evaluation; the other analyses use our own retests.

These retests run the RGB system on ten new layouts, layouts 10 to 19 of seed 0 (standard and cluttered layouts 5 to 9 for Generalization tasks), with the benchmark's checkers, the same prompt and the task tools on all 42 tasks. Swap blocks runs with the benchmark's fix of its button reset, and with tools developed again under that fix. The depth network results, and the comparisons in Secs. 4.3 to 4.5 and 5.2, use the ten development layouts, the first ten of seed 0 (five standard and five cluttered for Generalization tasks), with the bare instruction.

4.2 Main results

HexaShell was evaluated by RoboDojo itself and leads the official leaderboard (Fig. 2). Fig. 3 compares the task types with the other entries. In RoboDojo's official evaluation, HexaShell reaches 46.9% success and a progress score of 53.2 on the leaderboard's metric, above WorldScape-R^{2}SI (39.4%, 46.6), VPP2 (32.3%, 39.3), PhysicalRSI (31.4%, 36.3) and every other entry. By task type it reaches 34.5% on Generalization, 32.0% on Precision, 31.5% on Long-Horizon, 81.0% on Memory and 55.3% on Open. HexaShell leads where the task is decided by reading the scene and remembering it: on Memory and Open it is ahead of every entry, and the best learned model, WorldScape-R^{2}SI, reaches 63.7% and 47.5%. Learned policies lead where precise contact decides: VPP2 reaches 35.2% on Precision and 37.0% on Long-Horizon, against our 32.0% and 31.5%.

Over the 54 variants the plain mean is 41.9%, against 30.6% for VPP2, 26.1% for PhysicalRSI and 24.3% for the same model without tools. Per variant, HexaShell is higher than VPP2 on 28, by a median of 40 points, and lower on 20, by a median of 31 (Fig. 13). Two variants pass every episode, cover blocks and the standard variant of push T, and 11 stay at zero.

4.3 Clean versus cluttered layouts

The cluttered variants test what a system relies on, because the task is unchanged and only the scene around it is. In the official evaluation, HexaShell passes 38.0% of the standard and 31.0% of the cluttered Generalization episodes, keeping 82% of its success; on the development layouts it kept 95%. The same model without tools keeps 87%. The 31 learned policies keep 1–78%, with a median of 20%; VPP2 drops from 46.4% to 12.6%, and PhysicalRSI from 25.7% to 5.6%. The reason lies in what the tools compute: the agent picks a pixel on the target, and the tool measures that object and nothing else. Cloth is the exception that fits the rule: fold clothes falls from 56% to 24%, because its tool finds the garment by colour connectivity, which an object on or beside it breaks.

4.4 Triangulation versus a depth network

Tab. 1 compares three ways for the tools to get 3D. On the same layouts, triangulation scores 8.7 points higher than the depth network on the leaderboard's metric. Paired by layout, geometry turns 57 failures of the depth network into successes and loses 23 (exact McNemar test, $p < 0.001$). The largest gains are push T (4 to 10 of 10 layouts), imitate

Table 1. Triangulation against a depth network, same harness, loop and model. Success rate and progress score in %. Triangulation is our RGB tool set; Depth Anything V2 estimates depth from each image. Both are frozen retests on the development layouts. Rendered depth is not part of the standard observation. Last row: layouts that only one of the two passes, of 420.

	Triangulation		Depth Anything		Rendered depth	
Type	Succ.	Prog.	Succ.	Prog.	Succ.	Prog.
Generalization	34.2	37.7	26.7	30.0	45.4	51.5
Precision	21.3	28.4	7.5	11.9	41.3	47.3
Long-Horizon	18.8	32.2	13.8	29.2	32.5	44.2
Memory	73.3	73.3	55.0	58.0	66.6	66.6
Open	43.8	46.6	45.0	48.1	58.5	60.7
Leaderboard score	38.3	43.6	29.6	35.4	48.8	54.1
Mean of 54 variants	35.7	40.8	27.8	33.0	47.2	52.8
Layouts only it passes	57		23			

Table 2. What second views cost, 420 retest episodes each on the development layouts. A viewing move is a base motion followed, before any other motion, by a measurement that reads a wrist camera, or one of three tools whose only job is to look. Viewing inside larger tools is not counted, so the first three rows are lower bounds.

	Triangulation	Depth network
Executed steps spent on viewing	3.6%	0.7%
Episodes with a viewing move	180 (43%)	40 (10%)
viewing steps per step limit, median	4.9%	2.9%
Successes	152	118
step limit used, median	70.0%	63.1%
Failures that used 75% of the limit or more	189 / 268	228 / 302
Episodes ended by the step limit	20	26
Successes on the 180 layouts with a viewing move	39	22
Successes on the other 240 layouts	113	96

sorting sequence (1 to 8) and play Xylophone (0 to 6); six more tasks gain three layouts each. The losses are smaller: pour liquid into cup, pour by language, stack bowls and arrange largest number lose two layouts each. Three of these four put something in or on a round container whose rim the tools must measure. Pour balls into vase also uses a round container, yet it gains three.

The reason is accuracy. Against rendered depth on 300 frames of eight tasks, the depth network's median error on object pixels was 41 mm at the head camera and about 100 mm at the wrists, after we fixed its scale with the table. Three other networks we tried, Depth Pro [5] and two versions of MoGe [60, 61], reduced the head error to 20–31 mm but left 21–72 mm at the wrists. Grasps and placements need about a centimetre. Geometry has no such floor: when its assumption holds, a known plane or a point visible in two views, the error comes from the calibration and the pixel. Rendered depth is still 10.6 points higher. Most of the gap is in tight insertion, discussed in Sec. 5.

4.5 The cost of a second view

Without depth, the tools need a second view more often, and the robot must move to make it. We count a viewing move as a base motion (move, point, rotate or home) that is followed, before any other motion, by a measurement that reads a wrist camera. We also count three tools whose only job is to look. Viewing stages inside larger tools are not counted, so the numbers are a lower bound. Tab. 2 gives the result. With triangulation, 3.6% of all executed steps go to viewing, against 0.7% with the depth network, and 43% of episodes contain at least one viewing move. In those episodes viewing takes a median of 4.9% of the step limit. Successful episodes use a median of 70% of the limit, against 63% with the depth network.

The price is worth paying. On the 180 layouts where the RGB agent made a viewing move, it succeeded 39 times; the depth network succeeded 22 times on the same layouts (22 against 5 discordant pairs, p=0.002). On the other 240 layouts the gap is smaller, 113 against 96. The steps did not make the budget tighter than with the depth network: fewer failures used three quarters of the limit (71% against 75%), and fewer episodes ended at the limit (20 against 26). Of the

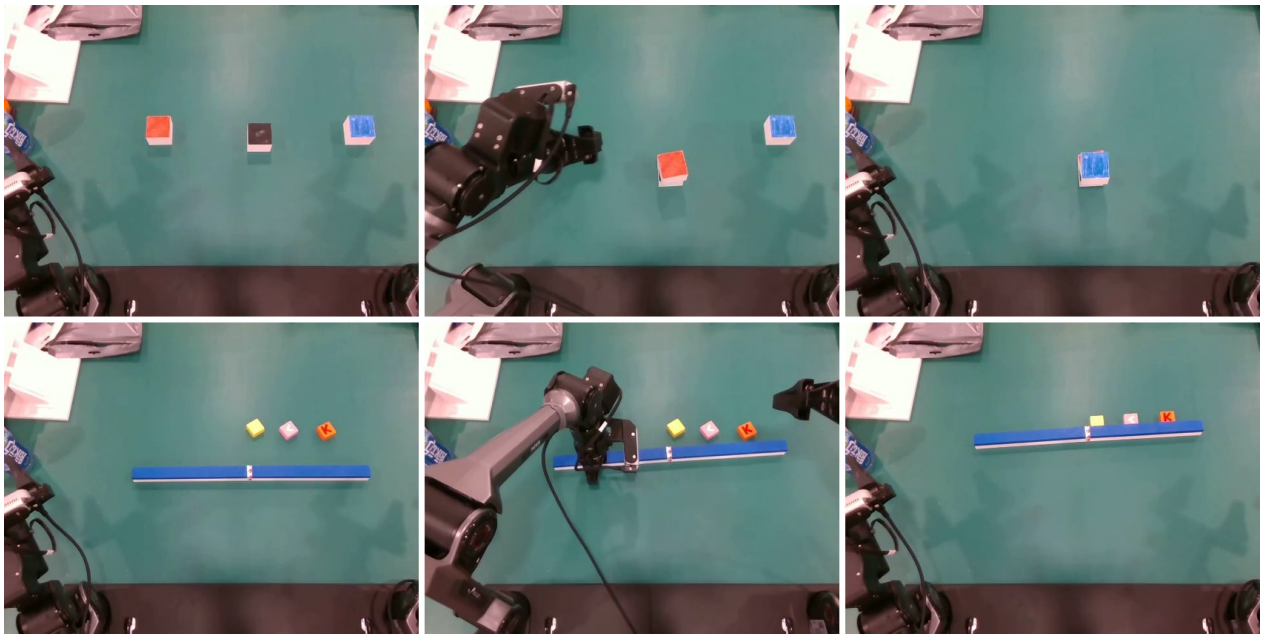


Figure 5. HexaShell on a real robot. A real setup with two PiPER X arms, built after RoboDojo's simulated scene, seen from above. Top: the robot stacks three blocks into one tower. Bottom: it lines up three letter blocks along a bar. Left to right: start, during the episode, end.

189 failures that used three quarters of the limit, 100 contained viewing moves, and only 21 would have stayed below three quarters without them. The cost is concentrated. It is highest where the tools must now measure what earlier tools had taken as fixed, such as an egg carton (40% of its steps) or the slot for a coin (16%), and none of these tasks succeeds. Where the view pays, it pays a lot: play Xylophone spends 17% of its steps on viewing and goes from 0 to 6 layouts, and push T spends 8% and goes from 4 to 10.

4.6 What the agents built

The RGB runs took 769 optimizer rounds (median 17 per task); a round after a failure took a median of 6.3 minutes. The runs left 327 task commands in about 78,000 lines of Python, counting the modules they import (Tab. 6). In the retests on the development layouts, 42 commands were called with a second camera, 998 times in 25 tasks. The measurements fall into a few patterns: a plane height from a camera pair, a point on a measured plane, a triangulated point, patch or contour, and a rim fitted in two views. The model picks the pixel and the code measures the world. This split keeps the tools easy to check, and, as Sec. 5 shows, it also sets their limits. Sec. C follows push T through its development.

4.7 On a real robot

At run time, the tools use only RGB images, the camera calibration and the state of the arms, and a real robot gives the same inputs. We built a real setup after RoboDojo's simulated scene, with two PiPER X arms and calibrated RGB cameras. The arm differs from the simulated ARX X5. After a small change for the new arm model, the perception and execution tools built in simulation run on the real robot, under the same command line. Fig. 5 shows two episodes. In the first, the robot stacks three blocks into one tower. In the second, it lines up three letter blocks along a bar. The move took little effort, because no tool depends on a signal that only the simulator has.

5 Where the boundary lies

The averages hide a sharp split. To find what separates the tasks we use the retests of the RGB tools, where 18 variants are at or above 50% and 19 are at 0%.

Table 3. Capability map and failure modes. Mean success over variants for the same model without tools (None, [69]), HexaShell with a depth network (DN, development layouts) and with triangulation (Ours, new layouts), and VPP2, the highest learned policy without an agent; RGB variants at 0% on the new layouts; and how failed RGB episodes on the development layouts ended: the agent *declared* completion and the checker disagreed, the budget was *exhausted* (less than a quarter left), or the agent *gave up* with more left.

		Success %					Failed episodes, %		
Class	Var.	None	DN	Ours	VPP2	At 0	Decl.	Exh.	Gave up
R rigid pick, place, stack	22	36.9	44.1	50.0	30.0	3	29	60	9
Pu push, slide, press	4	60.0	67.5	92.5	24.7	0	33	33	33
Mv moving objects	2	33.0	30.0	40.0	43.0	0	46	15	38
In tight insertion	12	1.7	1.7	8.3	28.1	7	32	53	15
Df deformable, fluid	6	26.0	30.0	33.3	49.4	2	15	77	8
Ar articulated, tool use	8	2.0	0.0	7.5	21.6	7	30	60	9
G geometry only	30	22.3	23.0	32.0	35.3	13	31	54	15
S symbols, language	18	22.8	26.7	31.1	14.4	6	24	65	11
M memory	6	38.7	55.0	65.0	55.6	0	50	25	13
All	54	24.3	27.8	35.4	30.6	19	30	57	13

5.1 A capability map

We label every task twice (Tab. 5). The *physical* label names the contact that success requires: rigid pick, place or stack with centimetre tolerance (R); pushing, sliding or pressing (Pu); insertion or placement with millimetre or few-degree tolerance (In); articulated objects or a held tool used through several contacts (Ar); deformable, fluid or granular material (Df); and objects that move on their own (Mv). The *decision* label names what must be decided beyond geometry: nothing (G), reading symbols, following language or recognising categories (S), or remembering an earlier state (M). Each task has one of each.

The physical interaction decides (Tab. 3). Pushing reaches 92.5% and rigid manipulation 50.0%; moving objects reach 40.0% and deformable material 33.3%, articulated objects 7.5%, and tight insertion 8.3%, with 7 of its 12 variants at zero. The decision load matters less, except for memory (65.0%). VPP2 has the opposite profile: it is weaker on pushing (24.7%) and rigid manipulation (30.0%), and stronger on insertion (28.1%), deformable material (49.4%) and articulated objects (21.6%).

What the tools add is the gap to the column without tools. They add most where the right action can be computed from measured geometry: pushing gains 33 points, rigid manipulation 13 and memory 26. Articulated objects gain a little, mostly from play Xylophone (6 of 10 layouts); these are the first articulated successes in any of our runs. On cloth the tools still do worse than an agent that corrects step by step: fold clothes reaches 50% against 72% without tools. A tool fixes one interpretation and one motion planned in advance; an agent acting step by step can still notice and correct.

Tight insertion and pouring are where measuring without depth costs most. With rendered depth, the tools reach 30.1% on insertion, led by insert key (80%) and play stacking toy (73%), while the RGB tools reach 8.3%. Pouring falls as far: pour liquid into cup and pour by language drop from 59–95% to 0–10%. An insertion needs the hole and the part in the same frame to a few millimetres. At the head camera one pixel covers 2–3 mm on the table, and the wrist views that would resolve the hole must be made while the gripper holds the part.

5.2 How episodes fail

Of the 420 retest episodes on the development layouts, 268 failed. Five of them ended early on a tool error; of the other 263, 57% exhausted the budget, 30% ended with the agent declaring completion and the checker disagreeing, 13% ended with the agent giving up while a quarter or more of the budget remained, and two were stopped by the checker for a rule violation.

The mix differs by class (Tab. 3), and the difference is diagnostic. On deformable material, articulated objects and rigid tasks, 60–77% of the failures exhaust the budget: the agent sees that the result is wrong and retries until the steps run out, so it knows it has failed and cannot make the motion work. On memory tasks half of the failures are declared successes: the agent believes it has reproduced the earlier state, and the checker disagrees. This fits the known weakness

of vision language models at fine visual detail [24, 51] and at judging success [20]. Moving objects often show a third signature: the agent gives up once the object it was waiting for has passed. A declared failure therefore points to a gap in perception or in the task's specification, an exhausted one to a gap in execution.

5.3 Missing information or missing capability?

In an earlier run with rendered depth, short notes on what the checker requires lifted three tasks from 0–40% to 95–100% (Tab. 7). The notes stayed available in the RGB runs. On the development layouts, press by number keeps 9 of 10 layouts and pour balls into vase 7, but pour liquid into cup falls to 1, while spending a fifth of its steps on viewing. Knowing what the checker wants is necessary but not enough: the tool must also measure the cup, and without depth it is still too weak.

6 Case studies: from a failed attempt to a working tool

The results above come from frozen tools. This section shows how the tools got there. Each case follows one layout through one turn of the loop of Sec. 3.4: an attempt that fails, the optimizer round that follows, and the next attempt on the same layout with the edited tool.

6.1 What an optimizer round looks like

The RGB runs took 769 optimizer rounds. Of these, 530 followed a failure, 195 followed a success and recorded what had worked, and 42 closed a run. A round after a failure took a median of 6.3 minutes, and 90% of them took less than 11. In that time the optimizer ran a median of 16 shell commands. It reads its role and the focus file of the round, the failure table, the command log of the failed episode, the acting agent's transcript and the code of the enabled tools. It then edits a tool, writes or updates unit tests and runs them; 92% of these rounds ran the task's tests. The median edit added 118 lines and removed 6, tests and notes included. A round read a median of 0.75 million input tokens, most of them cached, and wrote 5.7 thousand. Over all rounds the optimizer read 487 million input tokens and wrote 3.8 million.

In 27 tasks a layout failed and then passed on the next attempt after an edit, 67 times in all. For each candidate we checked in the log of the successful attempt that the edited code ran and that it addressed what had failed, and for two cases we ran the old and the new code on the stored images. Tab. 4 lists the seven cases we kept, at least one for each physical class of Tab. 3. In five of them the optimizer looked at no image: the command log, the tool's feedback and the code were enough. In fold clothes it ran its code on stored test images, and only in swap T did it analyse a frame of the failed episode.

6.2 Push T: a height check that compared unequal pixels

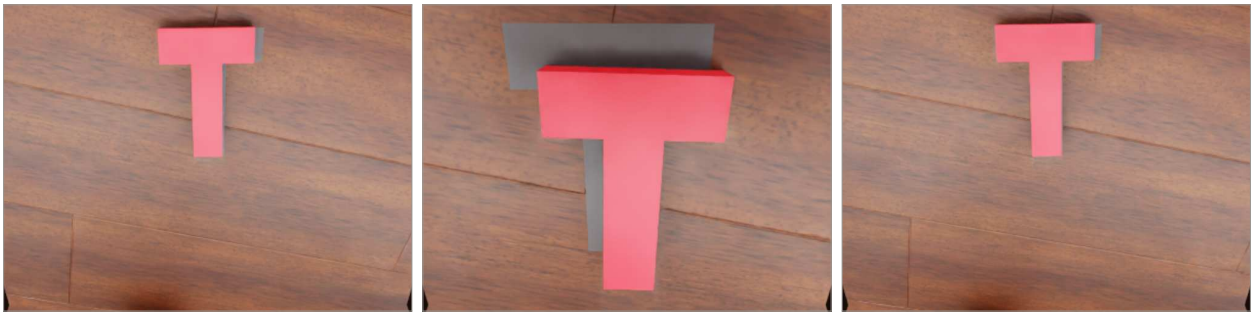
In push T the robot pushes a T block onto a T pad. The block must end within 7 mm and 7 degrees of the pad, with both arms at home. The tools project the block's outline from the head camera onto a plane at the block's top, and a stereo command measures the height of that plane from two cameras. On layout 2 in round 4 the height command refused all ten requests that reached its area check: it found the block's area in the two views too different. The agent fell back to the table height, 0.765 m, while the top of the block is at 0.780 m. The 15 mm error shifted the projected outline by 10 mm. The first transfer left the block 3.2 mm from the pad, inside the limit, but the release check used the same wrong height and reported 12.5 mm. The agent corrected three times and ended 13.6 mm off (Fig. 6).

The optimizer found the cause in the code, from the tool's error messages: "The stereo code estimates area by counting occupied grid cells; a distant camera can therefore report a smaller area than a close camera for the same surface." The coarse pixels of the head camera fill only about a quarter of the 1 mm cells that the close wrist camera fills. The optimizer replaced the count with the sum of each pixel's footprint on the plane. Its new test, with two cameras of different resolution, failed before the fix and passed after, and all 73 tests passed.

In round 5 the first height request returned 0.780 m, and the agent used it for every later step. The release check reported 1.6 mm, the true error was 3.0 mm, and the episode succeeded at step 354 of 600. On the stored images of that request the old code finds the same height but rejects it, with an area ratio of 0.27 against a lower bound of 0.85. With frozen tools layout 2 passed again in 251 steps, and push T passed all ten development layouts, standard and cluttered. Sec. C follows the whole push T run.

Table 4. Seven turns of the loop. Each row is one layout: the attempts that failed on it before the success (Fails), what the optimizer found after the last failure, what it changed, the size of the edit in lines added and removed (tests included) and the length of the round in minutes, the action steps of the next attempt, which succeeded, and the frozen retest of the same layout, with the task's retest over its ten development layouts.

Task	Layout	Class	Fails	Found	Changed	Edit	Min.	Next	Retest	
Push T	2	Pu	2	area check counted grid cells, so cameras with different pixel sizes disagreed; height refused	area from each pixel's footprint on the plane	+49/−11	7	354/600	pass	10/10
Swap T	0	R, M	2	colour mask mixed top and side faces; height failed seven times	match colour by hue, keep the bright top face	+75/−5	8	285/400	pass	10/10
Play Xylophone	1	Ar	4	one camera put the held mallet head 23 mm off	match a second view and triangulate	+109/−7	5	377/500	pass	6/10
Match and pick from conveyor	4	Mv	1	pick schedules shorter than the arm's motion; two aborts	estimate motion cost first, close later	+132/−5	6	587/700	pass	7/10
Play tic tac toe	0	R, S	3	last ring needed 146 steps, 84 left	turn the gripper while moving	+194/−5	11	1028/1100	pass	9/10
Fold clothes, cluttered	1	Df	1	grasp point 1.5 mm from the cloth edge	require 12 mm of cloth around the point	+112/−12	9	344/500	fail	6/10
Insert tubes	4	In	3	unmeasured voxels along camera rays blocked every path after the lift	recheck the original path from the reached joints	+69/−3	5	398/500	pass	3/10



(a) Round 4: the block is 3.2 mm from the pad; the check, at table height, reports 12.5 mm

(b) Round 4: after three corrections the block is 13.6 mm off; the agent quits

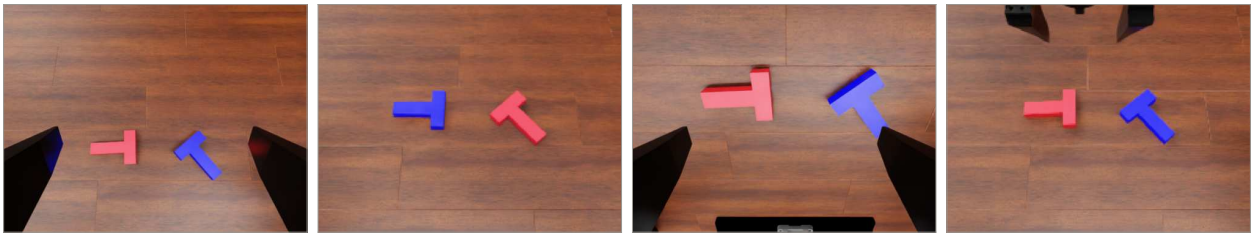
(c) Round 5: with the measured height the check reports 1.6 mm; success at step 354 of 600

Figure 6. Push T, layout 2: a height that the tool refused. Left wrist camera. (a) and (c) show almost the same placement; only the measured height of the block's top differs.

6.3 Swap T: a colour mask that mixed two faces

In swap T the robot exchanges two T blocks. Each block must end within 2 cm of the other's start and within 3 degrees of its heading. On layout 0, round 1 failed when a recovery turned a block twice, and the optimizer enabled a measuring tool, surface_pair, which fits the outlines of both blocks and returns the turn between them. In round 2 the acting agent

called surface_pair seven times, and each call failed to measure the block height. It then typed a height of 0.78 m, and the tool returned a turn of 130.6 degrees with an outline fit error of 2.6 mm. Both blocks ended within 5 mm of their targets, and the check failed on their headings (Fig. 7).



(a) Round 2: the dark side faces share the hue of the top faces

(b) Round 2: both blocks within 5 mm, headings about 3° off; check fails

(c) Round 3: only the bright top faces are kept; outline fit 0.4 mm

(d) Round 3: the tool measures the height itself; success at step 285 of 400

Figure 7. Swap T, layout 0: two faces of one colour. (a, c) left wrist camera; (b, d) head camera, cropped.

The optimizer turned to the angle. It clustered the colours of the last head frame and found that each block splits into a bright top face and a darker side face of the same hue: “I’m investigating whether the color masks mixed top and side faces, which would distort both height and rotation estimates.” It also found that the tool matched one fixed colour across the cameras, while each camera sees the blocks under different light. It built a synthetic scene with both effects, and the old tool failed on it with the episode’s error. It changed 28 lines of surface_pair to match colours by hue and to keep only the bright top face in each view. All 80 tests passed.

In round 3 surface_pair measured the height itself, 0.781 m, and returned a turn of 126.2 degrees with an outline fit error of 0.4 mm. The swap passed at step 285 of 400. Each block of round 2 had ended about 3 degrees from where it ended in round 3. With frozen tools layout 0 passed again in 275 steps, with a measured turn of 126.3 degrees, and the task passed all ten development layouts.

6.4 Play Xylophone: a second view of a held tool

In play Xylophone the robot picks up a mallet and strikes the eight keys from left to right. The checker counts a key when the mallet head is inside its box, 1.0 to 3.6 cm above it, and lifted at least 2.5 cm before the next key. The task’s note states these numbers, and the tool’s default heights follow them. Layout 1 failed four times, each time for a different reason: plans rejected for clearance, a ball outline that was not round, a grasp on the stand instead of the mallet, and in round 9 the position of the held mallet. In round 9 the acting agent lifted the mallet and measured its head with rgb_sphere from the head camera alone, using the radius of the head to set the range. The estimate was 22.9 mm from the true centre. Two sequence plans failed the reach check, so the agent moved all eight targets 5 to 8 cm toward the near ends of the keys. The third plan ran all eight strikes, and the checker counted none (Fig. 8).



(a) Round 9: one camera puts the mallet head 23 mm from its true centre

(b) Round 9: eight strikes, shifted to the near ends of the keys; no key counts

(c) Round 10: the right wrist sees the whole head; two views put it 2.7 mm off

(d) Round 10: strikes at the measured key centres; success at step 377 of 500

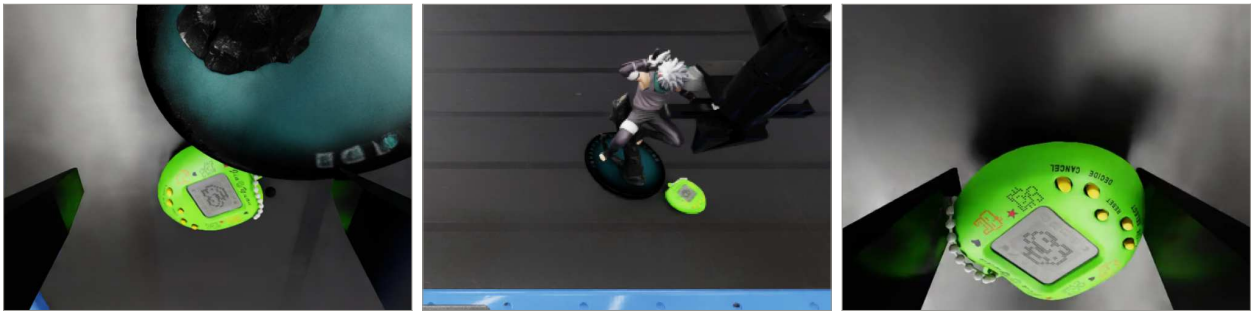
Figure 8. Play Xylophone, layout 1: one view against two. (a) head camera, cropped; (b–d) right wrist camera.

The optimizer traced the miss to the estimate of the held mallet head from one camera, and changed rgb_sphere. Given one pixel on the ball, the tool now finds the matching ball in another current camera and triangulates the centre. The radius only filters the candidates, and without a clear match the tool fails instead of guessing. In its new synthetic test the old method erred by 22.7 mm and the new one by less than 2 mm. All 35 tests passed.

In round 10 the first call failed, because the ball was cut off at the border of the wrist view. The agent moved both arms until the right wrist camera saw the whole head. The tool matched the two views and put the centre 2.7 mm from the true one, with 0.7 mm between the two rays. The sequence ran at the measured key centres on its first plan, and the task succeeded at step 377 of 500. In the frozen retest layout 1 passed again, this time with strikes made by base moves, and the task passed 6 of its 10 development layouts. The depth network version passed none.

6.5 Match and pick from conveyor: a schedule the arm could not meet

In match and pick from conveyor the robot remembers the first object on a moving belt and picks the matching object when it comes back. On layout 4 in round 9 the tools located the green toy to within 3 to 11 mm. Twice the agent asked the tool timed_pick for a short schedule, with the descent due 0.80 s after the call. Turning and moving the arm took 1.56 s, so both times the tool aborted before it went down, and the open fingers waited above the passing toy. A manual grasp then closed 67 mm behind it (Fig. 9).



(a) Round 9: the pick aborts before descent; the open fingers wait above the passing toy

(b) Round 10: the head view the agent measures the toy in before its last pick

(c) Round 10: the grasp moves to 2.16 s; the toy is lifted at step 587 of 700

Figure 9. Match and pick from conveyor, layout 4: time, not position. (a, c) right wrist camera; (b) head camera, cropped. (c) is the last frame of the video, since the episode ended during the lift.

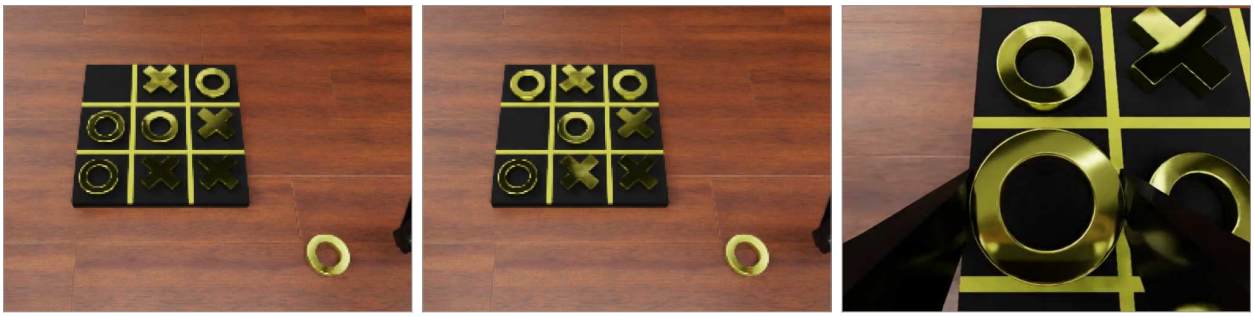
The optimizer saw that vision was fine and the schedules were impossible: “The trace also shows two approaches that consumed motion time before reporting a missed deadline, plus a completed pick that did not retain the object.” It found a server call that estimates the cost of a motion without moving the arm, and added a check to timed_pick. Before moving, the tool now estimates each stage. If the schedule is too short, it moves the closing time later and the meeting point further down the belt. All 20 tests passed.

In round 10 the agent again asked for a short pick, closing at 1.5 s. Getting into place took 1.68 s, so the old tool would have aborted again. The new check moved the closing time to 2.16 s and the meeting point 64 mm further along the belt. The arm reached the toy as its approach ended, closed on it and lifted it, and the task succeeded at step 587 of 700. In the frozen retest layout 4 passed in 460 steps, and the task passed 7 of its 10 development layouts.

6.6 Play tic tac toe: a route that saves steps

In play tic tac toe the robot plays first and fills the board with a scripted opponent: five rings from the robot, four crosses from the opponent’s arm, within 1100 action steps. On layout 0 in round 3 the first four rings landed within 6 mm of their cells, but these placements used 1016 steps. Waiting for the opponent took 520 of them. Before each transfer the tool turned the empty gripper at the home pose, at 16 to 22 steps a time. The first entry also stopped 8.1 mm short of its depth, over the 8 mm limit, and cost 56 steps to redo. The route for the last ring needed 146 steps when 84 were left, and the agent sent both arms home with one cell empty (Fig. 10).

The optimizer read the step count of every transfer from the log. It kept the waits, which the game requires, and went after the turns: “The transfer tool rotates the empty gripper at home before moving above the source. I’ll check whether combining those two motions can save enough steps while preserving vertical contact moves and clearance.” The transfer now also plans a route that turns the gripper while it moves above the ring, keeps the fingertips above the travel height during the turn, and takes the cheaper reachable route. The optimizer added eight tests, and all 192 passed.



(a) Round 3: eight cells filled, the last ring left at its source; steps run out

(b) Round 4: same position with 140 steps left; the last route needs 129

(c) Round 4: the fifth ring goes into the last cell at step 1028 of 1100

Figure 10. Play tic tac toe, layout 0: the step budget. (a, b) head camera, cropped, before the last ring; (c) right wrist camera, last frame of the video.

In round 4 all five transfers took the combined route. The first placement fell from 152 to 128 steps, and the next three from 344 to 312. The first entry stopped inside the depth limit this time, and the cell order left a nearer cell for the last ring. Before the last ring 140 steps remained, and its route needed 129. The board filled at step 1028 of 1100. In the frozen retest layout 0 passed at step 1035, and the task passed 9 of its 10 development layouts.

6.7 Fold clothes: a grasp point on the edge

In fold clothes the robot folds both sleeves of a shirt across the body and then makes a last fold. The cluttered variant places other objects around it. On layout 1 in round 13 the landmark tool cloth_view placed the grasp point of the left sleeve 1.5 mm from the cloth edge, and the gripper pinched only the edge of the cuff. The left sleeve stayed out. The acting agent saw this, grasped the sleeve again at its middle and left it bunched on its own side. The last fold scored nothing, and the episode ended at step 453 of 500 (Fig. 11).



(a) Round 13: the grasp point is 1.5 mm from the edge; the left sleeve stays out

(b) Round 13: a second grasp bunches the sleeve; the last fold scores 0

(c) Round 14: grasp points 12 mm inside the edge; the sleeve crosses in one move

(d) Round 14: both sleeves cross; the last fold succeeds at step 344 of 500

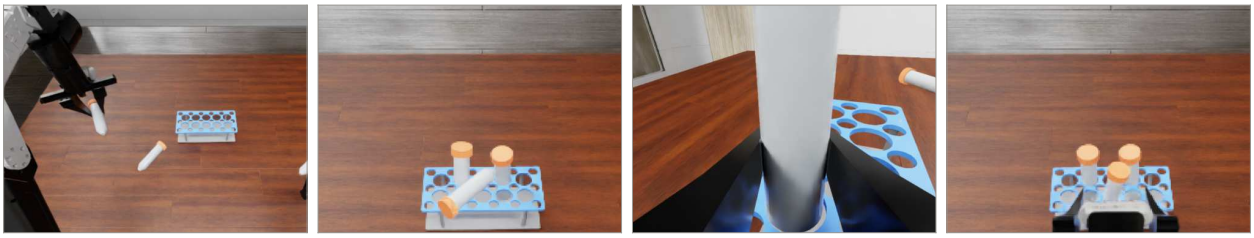
Figure 11. Fold clothes, cluttered layout 1: where to grasp a sleeve. Head camera, cropped, after the sleeve folds (a, c) and after the repair grasp of round 13 (b) and the crossed sleeves of round 14 (d).

The optimizer read the transcript and the tool's feedback, then ran the grasp point code on four stored test images with their calibration and measured each point's distance to the cloth edge: "On four archived RGB fixtures, lateral contacts sit only 0.9–7.4 mm from an edge despite the nominal 25 mm inset." The cause was a check that accepted a point when one pixel of it fell on the cloth, and returned an unchecked point when it found none. The optimizer replaced it with a test on the table plane that requires 12 mm of cloth on every side of the point, searched on a 2 mm grid up to 6 cm inward. All 37 tests passed.

Between rounds 13 and 14 only cloth_view changed. In round 14 it returned points 12.3 and 12.1 mm inside the edge; on the same frame the old code returns points 1.6 and 5.4 mm from it. The left sleeve crossed the body in one move, no repair was needed, and the shirt was folded at step 344 of 500. In the frozen retest this layout failed with a progress of 20%, and the task passed 6 of its 10 development layouts.

6.8 Insert tubes: an early stop in the hardest class

In insert tubes the robot places three tubes upright in the holes of a rack. Insertion is the class where the RGB tools do worst (Sec. 5.1), and layout 4 failed in rounds 16, 17 and 18. In round 18 the agent grasped the first tube and called rigid_place. The arm lifted the tube and stopped: no path to the rack was clear. After the lift, stereo found nothing near the path, so the tool fell back on single camera rays and marked 28,935 voxels along them as obstacles, although their depth was never measured. Both alternative paths failed inverse kinematics, and the tool gave up, although the original path had passed its check before the lift. Recovery by hand cost 108 of the 500 steps. A later tube was released after a 5.7 mm lowering error and fell across the rack, and the agent quit at step 449 (Fig. 12).



(a) Round 18: after the lift the tool stops; recovery costs 108 steps

(b) Round 18: a later tube lies across the rack; the agent quits

(c) Round 19: the recheck passes; the tube ends 1.5 mm from the hole centre

(d) Round 19: all three tubes stand in the rack at step 398 of 500

Figure 12. Insert tubes, layout 4: a stop after the lift. (a, b, d) head camera, cropped; (c) left wrist camera.

The optimizer read the tool's feedback and the command log: "The clearance fallback treated 28,935 voxels along monocular rays as a hard constraint, although their depths were unverified. It rejected both proposed yaw paths and stranded the grasp." It kept those rays as a preference and added one more check: when the preferred paths are rejected, the tool rechecks the original remaining path from the joints it has reached and goes on if that path is reachable. Obstacles seen in two views still block the move. All 183 tests passed. The optimizer noted that the change does not explain the sideways release.

In round 19 the same situation came up, with 27,626 ray voxels and two rejected paths. The recheck passed, and the first tube reached its hole 1.5 mm from the measured centre. Lowering stopped early, and the agent opened the gripper and let the tube drop in. The first tube took 64 steps instead of 108, the other two needed moves by hand, and the episode succeeded at step 398 of 500. In the frozen retest layout 4 passed in 411 steps without needing the recheck, and the task passed 3 of its 10 development layouts. The other seven ended with the agent quitting or at the step limit.

6.9 What the cases show

The optimizer works like a programmer reading a bug report. Its evidence is mostly numbers: the step count of each command, the reason a tool gave for refusing. Its edits are small and local. Each changes one tool, comes with new tests, and leaves the strategy to the acting agent. Three of the seven fix a measurement (push T, swap T, play Xylophone), two fix timing (the conveyor schedule and the tic tac toe route), one fixes where to grasp (fold clothes) and one fixes when a tool gives up (insert tubes). In insert tubes the fix removed one obstacle, and the tube still went in by a drop. This matches the boundary of Sec. 5: a round of the loop can repair what the agent measures and plans, but not the contact that tight insertion needs.

7 Conclusion

HexaShell gives an unchanged coding agent one command line to a robot and keeps what the agent learns as small tools, repaired after each failure in a round of a few minutes. With triangulation from calibrated RGB cameras, the tools locate objects more reliably than a depth network, and in RoboDojo's official evaluation on a hidden seed of all 42 tasks the agent reaches 46.9% success and a progress score of 53.2. Because at run time the tools see only what a real robot also sees, they move to a real robot with a different arm after a small change for the arm. The limit is contact, not reasoning: pushing and memory tasks succeed most of the time, but tight insertion stays low, and most failed episodes run out of steps while the agent sees that the motion is not working. Closing this gap is the next step: tools that keep measuring while the gripper holds the part and correct the motion during contact, instead of planning it in advance.

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2022.

[2] John Aloimonos, Isaac Weiss, and Amit Bandyopadhyay. Active vision. *International Journal of Computer Vision*, 1(4):333–356, 1988.

[3] Ruzena Bajcsy. Active perception. *Proceedings of the IEEE*, 76(8):966–1005, 1988.

[4] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[5] Aleksei Bochkovskii, Amaël Delaunoy, Hugo Germain, Marcel Santos, Yichao Zhou, Stephan R. Richter, and Vladlen Koltun. Depth pro: Sharp monocular metric depth in less than a second. In *International Conference on Learning Representations (ICLR)*, 2025.

[6] Gary Bradski. The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 2000.

[7] Anthony Brohan, Noah Brown, Justice Carbajal, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[8] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large Language Models as tool makers. *arXiv preprint arXiv:2305.17126*, 2024.

[9] Boyuan Chen, Zhuo Xu, Sean Kirmani, Brian Ichter, Danny Driess, Pete Florence, Dorsa Sadigh, Leonidas Guibas, and Fei Xia. SpatialVLM: Endowing vision-language models with spatial reasoning capabilities. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[10] Junting Chen, Yao Mu, Qiaojun Yu, Tianming Wei, Silang Wu, Zhecheng Yuan, Zhixuan Liang, Chao Yang, et al. RoboScript: Code generation for free-form manipulation tasks across real and simulation. *arXiv preprint arXiv:2402.14623*, 2024.

[11] Kaiyuan Chen, Shuangyu Xie, Letian Fu, Justin Yu, William Pacini, Sandeep Bajamahal, Hudson Kim, Jaimyn Drake, Daehwa Kim, Haoru Xue, et al. GaP: A graph-as-policy multi-agent self-learning harness for variational automation tasks. *arXiv preprint arXiv:2607.05369*, 2026.

[12] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, et al. Evaluating Large Language Models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[13] Tianxing Chen, Zanxin Chen, Baijun Chen, et al. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.

[14] Tianxing Chen, Yue Chen, Zixuan Li, Junyuan Tang, Kailun Su, Haoran Lu, Weijie Wan, Baijun Chen, Songling Liu, Haowen Yan, et al. RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026. Leaderboard at `https://robodojo-benchmark.com`, accessed October 2026.

[15] Yanzhe Chen, Zechen Bai, Zhijun Cao, Wenzheng Zeng, Kevin Qinghong Lin, Yiqi Lin, Guoqiang Liang, Kevin Yuchen Ma, Qiming Huang, and Mike Zheng Shou. Show-Harness: Just a VLM agent can play robots. *arXiv preprint arXiv:2609.10522*, 2026.

[16] An-Chieh Cheng, Hongxu Yin, Yang Fu, Qiushan Guo, Ruihan Yang, Jan Kautz, Xiaolong Wang, and Sifei Liu. SpatialRGPT: Grounded spatial reasoning in vision language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[17] Robert T. Collins. A space-sweep approach to true multi-image matching. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 358–363, 1996.

[18] Zijie Diao, Yitong Chen, Sicheng Xie, Tianyi Lu, Wujian Peng, Guojin Zhong, Houze Xu, Ziyi Ye, Zuxuan Wu, and Yu-Gang Jiang. LIBERO-Agent: Evaluating general-purpose agents for direct embodied manipulation. *arXiv preprint arXiv:2609.39507*, 2026.

[19] Xin Ding, Liang Mi, Mingzhe Huang, Zixuan Wang, Chao Zhang, Zixu Hao, Fu Chen, Xiangyu Li, et al. Zetta: An efficient closed-loop embodied harness for self-evolving physical intelligence. *arXiv preprint arXiv:2608.16590*, 2026.

[20] Yuqing Du, Ksenia Konyushkova, Misha Denil, Akhil Raju, Jessica Landon, Felix Hill, Nando de Freitas, and Serkan Cabi. Vision-Language models as success detectors. *arXiv preprint arXiv:2303.07280*, 2023.

[21] Karim Elmaaroufi, Justin Svegliato, Sarunas Kalade, Graham Schelle, Sanjit A. Seshia, and Matei Zaharia. RHO: Your coding agent is secretly a roboticist. *arXiv preprint arXiv:2606.16458*, 2026.

[22] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, et al. LIBERO-Plus: In-depth robustness analysis of Vision-Language-Action models. *arXiv preprint arXiv:2510.13626*, 2025.

[23] Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, et al. CaP-X: A framework for benchmarking and improving coding agents for robot manipulation. *arXiv preprint arXiv:2603.22435*, 2026.

[24] Xingyu Fu, Yushi Hu, Bangzheng Li, Yu Feng, Haoyu Wang, Xudong Lin, Dan Roth, Noah A. Smith, Wei-Chiu Ma, and Ranjay Krishna. BLINK: Multimodal Large Language Models can see but not perceive. *arXiv preprint arXiv:2404.12390*, 2024.

[25] Hongcheng Gao, Jingjing Zhou, Zelin Zheng, Shijia Ge, Jay Zhu, Yazhe Wang, Jianshu Zeng, Xuan Shangguan, Di Wu, Lingyu He, Zhiqi Jia, Sihang Wu, and Xiao He. Self-evolving coding agents: From digital programs to physical-world intelligence, 2026. arXiv preprint arXiv:2609.35432.

[26] Shijia Ge, Alex Zhou, Jianshu Zeng, Yexing Wan, Di Wu, Zelin Zheng, Yazhe Wang, Zhiqi Jia, Xuan Shangguan, Jay Zhu, Yijun Liu, Lingyu He, Sihang Wu, Xiao He, and Hongcheng Gao. Make code as policy great again: Frontier agents write, call, and evolve robot tools. *arXiv preprint arXiv:2609.39018*, 2026.

[27] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, second edition, 2004.

[28] Hengyuan Hu, Jensen Gao, Priya Sundaresan, Satvik Sharma, Jeannette Bohg, and Dorsa Sadigh. VIA: Visual interface agent for robot control. *arXiv preprint arXiv:2607.11119*, 2026.

[29] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, et al. Inner Monologue: Embodied reasoning through planning with Language Models. *arXiv preprint arXiv:2207.05608*, 2022.

[30] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. VoxPoser: Composable 3d value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, 2023.

[31] Wenlong Huang, Chen Wang, Yunzhu Li, Ruohan Zhang, and Li Fei-Fei. ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation. *arXiv preprint arXiv:2409.01652*, 2024.

[32] Mengzhao Jia, Yang Lin, Xixin Zhang, Zhihan Zhang, Xiaobai Liu, and Meng Jiang. Agent as policy for robotic manipulation. *arXiv preprint arXiv:2609.12541*, 2026.

[33] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models resolve Real-World GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.

[34] Yifan Kang, Zihan Wang, Zhiwen Fan, and Bangya Liu. Encore: Few-shot agentic discovery of manipulation strategies. *arXiv preprint arXiv:2609.37359*, 2026.

[35] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[36] Xuanlin Li, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, et al. Evaluating Real-World robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.

[37] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[38] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks*, 2023.

[39] Fangchen Liu, Kuan Fang, Pieter Abbeel, and Sergey Levine. MOKA: Open-World robotic manipulation through mark-based visual prompting. *arXiv preprint arXiv:2403.03174*, 2024.

[40] Yuyao Liu, Jiayuan Mao, David Hsu, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. RAPID: Robot agentic programming from demonstrations. *arXiv preprint arXiv:2609.30249*, 2026.

[41] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

[42] Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, et al. ASPIRE: Agentic skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026.

[43] Haitong Ma, Chenxiao Gao, Rushi Qiang, Bo Dai, and Na Li. RLE-Bench: A qualifying exam for coding agents as robot learning engineers. *arXiv preprint arXiv:2609.34210*, 2026.

[44] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[45] Yao Mu, Junting Chen, Qinglong Zhang, Shoufa Chen, Qiaojun Yu, Chongjian Ge, Runjian Chen, Zhixuan Liang, et al. RoboCodeX: Multimodal code generation for robotic behavior synthesis. *arXiv preprint arXiv:2402.16117*, 2024.

[46] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. RoboCasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.

[47] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

[48] NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

[49] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[50] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, et al. $\pi_{0.5}$: A Vision-Language-Action model with Open-World generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[51] Pooyan Rahmanzadehgervi, Logan Bolton, Mohammad Reza Taesiri, and Anh Totti Nguyen. Vision Language Models are blind: Failing to translate detailed visual features into words. *arXiv preprint arXiv:2407.06581*, 2024.

[52] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[53] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[54] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. ProgPrompt: Generating situated robot task plans using Large Language Models. *arXiv preprint arXiv:2209.11302*, 2022.

[55] Balakumar Sundaralingam, Siva Kumar Sastry Hari, Adam Fishman, et al. cuRobo: Parallelized collision-free minimum-jerk robot motion generation. *arXiv preprint arXiv:2310.17274*, 2023.

[56] Alkis Sygkounas, Victor Aregbede, Amy Loutfi, and Andreas Persson. MEMENTO: Memory-guided memetic code-as-policy evolution. *arXiv preprint arXiv:2607.22832*, 2026.

[57] Raman Talwar, Elias Nijs, Andreas Verleysen, and Francis wyffels. Generalizing manipulation skills with a local coding agent. *arXiv preprint arXiv:2609.26499*, 2026.

[58] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[59] Qi Wang, Tianyi Wang, Chengyang Li, Shikun Ban, Yurun Chen, Yizhong Ge, Jason Qin, Chengtai Li, and Wentao Zhu. Towards the harness of embodied agents. *arXiv preprint arXiv:2608.11246*, 2026.

[60] Ruicheng Wang, Sicheng Xu, Cassie Dai, Jianfeng Xiang, Yu Deng, Xin Tong, and Jiaolong Yang. MoGe: Unlocking accurate monocular geometry estimation for open-domain images with optimal training supervision. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[61] Ruicheng Wang, Sicheng Xu, Yue Dong, Yu Deng, Jianfeng Xiang, Zelong Lv, Guangzhong Sun, Xin Tong, and Jiaolong Yang. MoGe-2: Accurate monocular geometry with metric scale and sharp details. *arXiv preprint arXiv:2507.02546*, 2025.

[62] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.

[63] Wenli Xiao, Jia Xie, Tonghe Zhang, Haotian Lin, Letian Fu, Haoru Xue, Jalen Lu, Yi Yang, et al. ENPIRE: Agentic robot policy self-improvement in the real world. *arXiv preprint arXiv:2606.19980*, 2026.

[64] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[65] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything V2. In *Advances in Neural Information Processing Systems*, 2024.

[66] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montse Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, et al. Language to rewards for robotic skill synthesis. *arXiv preprint arXiv:2306.08647*, 2023.

[67] Lifan Yuan, Yangyi Chen, Xingyao Wang, Yi R. Fung, Hao Peng, and Heng Ji. CRAFT: Customizing LLMs by creating and retrieving from specialized toolsets. *arXiv preprint arXiv:2309.17428*, 2024.

[68] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.

[69] Wenbo Zhang, Kaixuan Wang, Yutao Ouyang, Xiaoyu Huang, Liyang Li, Kailun Su, Weiyang Jin, Wenhao Chai, Haotian Liang, Zhiyang Dou, et al. An unexpected robot policy: Early evaluations of GPT-6 Astra on RoboDojo and beyond. *arXiv preprint arXiv:2609.24170*, 2026.

[70] Yehang Zhang, Haojian Huang, Yifan Chang, Jianchong Su, Bohan Zhou, Yingjie Xu, Wosong Chen, Tianhao Zhou, Chenxu Wang, Tianyi Zhang, et al. World action agent: Harnessing VLMs for robot manipulation via world action rehearsal. *arXiv preprint arXiv:2609.29964*, 2026.

[71] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.

[72] Zhiyuan Zhou, Pranav Atreya, You Liang Tan, Karl Pertsch, and Sergey Levine. AutoEval: Autonomous evaluation of generalist robot manipulation policies in the real world. *arXiv preprint arXiv:2503.24278*, 2025.

[73] Di Zhu, Ziheng Yan, and Fang Wan. ActionUNet: Improving robustness of VLA models with efficient multi-scale fine-tuning. *arXiv preprint arXiv:2609.34982*, 2026.

Appendix

A Per-task results

Tab. 5 lists every task with its capability labels, RoboDojo's official evaluation, the retests with triangulation and with the depth network, the result of the earlier tools with rendered depth and two leaderboard entries. Fig. 13 shows the success rate of every variant next to VPP2 and PhysicalRSI. Fig. 14 compares the progress score per variant with the depth network version and with the same model without tools.

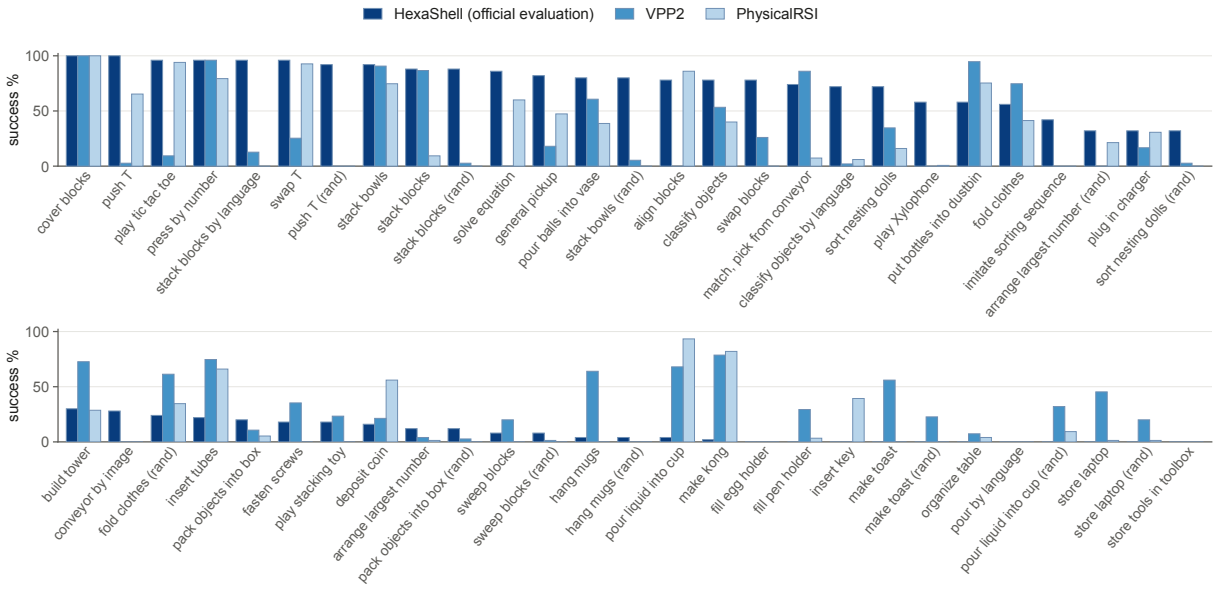


Figure 13. Success rate on every task variant: HexaShell against VPP2 and PhysicalRSI. Variants are sorted by HexaShell's success rate; rand: cluttered layouts. HexaShell: RoboDojo's official evaluation on a hidden seed, 50 episodes per task (25 per Generalization variant); VPP2 and PhysicalRSI: leaderboard rollouts, 50 episodes per seed.

B Tools of the RGB runs

Tab. 6 lists, for every task, the commands of the frozen RGB tools, the code behind them, the optimizer rounds it took and the share of executed steps spent on viewing moves (Sec. 4.5).

C Example development log: push T

The RGB run of push T took 13 optimizer rounds and four hours over its ten development layouts. It started from a tool of an earlier run, planar_contact, which registered the T block against its target outline in the head depth image and carried it there. Without depth this registration failed, but the acting agent passed layouts 0 and 1 anyway, measuring heights by hand with base commands and the shared table tool. Layout 2 failed twice. In round 3 the optimizer rewrote the registration, the height measurement, the handover between the arms and the release check to use RGB silhouettes and calibrated rays. In round 4 it fixed the check that compares the block's area in two views, which rejected matching views because the two cameras have different pixel scales. From then on every layout passed at the first attempt, standard and cluttered. Rounds 5 to 12 only added notes to the playbook, and round 13 consolidated them. The frozen retest passed all ten layouts, using 245 to 570 of the 600 steps, against 4 of 10 for the depth network version.

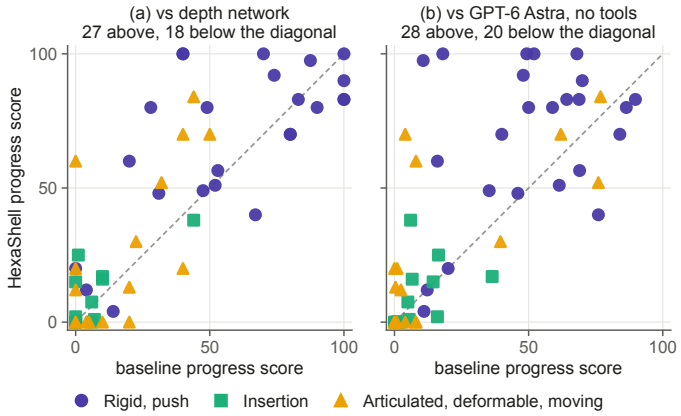


Figure 14. Progress score per task variant on the development layouts, HexaShell with triangulation against (a) the depth network version and (b) the same model without tools. Points above the diagonal favour triangulation. Colour and shape give the physical class (Sec. 5.1): rigid and push (R, Pu), insertion (In), and articulated, deformable or moving (Ar, Df, Mv). Means over the 54 variants: 40.8, 33.0 and 29.8.

D The earlier depth runs and the audit

The RGB tools grew out of two earlier lines. The first used rendered depth. Its fully automatic first round, on all 42 tasks, reached 38.0% on the leaderboard's metric. A second round gave the optimizer short notes on what the checker requires for the tasks where it lagged most (Tab. 7), and a team member rebuilt the tool of build tower by hand. Its current tools reach 48.8% (Tab. 1). The second line kept these tools but replaced rendered depth with Depth Anything V2 Base [65] inside the server. We developed 22 tasks again for up to 24 hours each, kept the new tools of ten of them, and the line reached 29.6%. The RGB runs started from the tools of the second line.

The notes of the first line stayed in the task directories and were given to the optimizer in every round. By the time of the RGB runs, 25 tasks had such a note, written by us from the public task definitions. They state the success conditions and, for some tasks, object sizes or target positions from these definitions. The acting agent never sees them. Nine tasks also started from RGB tools written by a colleague with another coding agent.

After the RGB runs we audited every enabled tool for numbers that could come from the benchmark's assets, layouts or configuration, by comparing every literal in the tools with the sizes and positions in the task definitions and in the logs. The tools of seven tasks held fixed object geometry: part sizes and templates of fixed scene parts. These tasks are store tools in toolbox, pick from conveyor by image, insert key, hang mugs, fill pen holder, fill egg holder and deposit coin. With the fixed geometry they had passed 3 of their 70 retest layouts. We replaced each constant with a measurement made in the episode and retested the seven tasks without further development; all are 0/10.

E Interface given to the acting agent

The acting agent's manual is one Markdown file, the same for every task; the interface.md of each enabled task tool is appended to it under the heading "Extra tools for this task". It contains, in this order:

- World frame. One right-handed frame for both arms and all cameras, in metres, degrees and seconds; origin on the floor between the arm bases, $+x$ to the robot's right, $+y$ away from it, $+z$ up; a nominal table top at $z{=}0.74$ (the shared table_plane tool measures the real height, about 0.765); arm bases at $x{=}\mp 0.30$, $y{=}-0.45$; how the frame appears in the head image, with a top-view drawing.
- Gripper. The TCP is the midpoint between the fingertips; the tool frame's x is the approach direction and y the opening axis; gripper value 1 is fully open (8.8 cm), 0 closed, and only the commanded value is reported. A reach envelope: pointing straight down the TCP reaches about $y{=}0.00$ at table height, pointing down45 about $y{=}0.15$.
- Base commands. obs and status (free); move (translate the TCP along world axes, at most 0.20 m per axis), rotate

(about the TCP in the world or tool frame, at most 90° per axis), point (down, forward or down45, with a required opening axis), gripper, home, wait (at most 5 s), each costing simulation time; done, which ends the episode.

- Limits. 60 commands per episode; simulation time limited per task; "the simulation only runs while a command runs; thinking costs no simulation time".
- Feedback. One JSON object per command: the request, clipping and workspace flags, plan_ok with a failure reason (ik_unreachable, self_collision, table_collision, workspace_limit), the reached pose and its error, settled, the gripper value, the time used and left, the commands left, and episode_over; exit codes for argument errors, failed commands, a finished episode and client time-outs. Motion planning knows the arm and the table, not the objects.
- Observations. Head and two wrist RGB images (640×480) and state.json with both TCP poses, joint angles, gripper values, and each camera's intrinsics and camera-to-world extrinsics, followed by the formulas that project a world point to a pixel and turn a pixel into a ray. There is no depth; the manual names the shared table_plane tool and the two ways to fix a point on a ray, a measured plane or a second view.
- Rules. When the task is finished, home both arms and then call done; call done only when sure; read a failed command's output before deciding; object positions are not given and must be worked out from the images and the calibration.

F Optimizer role description

The optimizer receives a 62-line role description and, each round, a focus.md that selects one of three modes. In *failure* mode it reads the failure category, every command with its target, the reached pose, the acting agent's trajectory and the head-camera frames; decides which perception tool, execution tool or repair would have prevented the failure, or that the failure was bad luck and nothing should change; implements it under tasks/⟨task⟩/tools/; enables it; and appends a dated entry to skill.md. In *success* mode it appends at most 15 lines to playbook.md and changes no tool unless one misbehaved. In *final* mode it rewrites playbook.md and skill.md (under 60 lines each) and every interface.md (under 12 lines). Six rules apply throughout: a tool may use only the EpisodeAPI (joint angles, TCP poses, motion primitives, the latest observation, the caller's arguments) and must never read simulator state or hard-code layout-specific numbers; positions come from OpenCV geometry on the RGB images and the calibration, and no size, height or position of an object may come from the benchmark's assets, layouts, configuration or logs; an interface.md describes the interface only and may not name the task or its objects, which the delivery check enforces with a list of banned words; the shared pool is read-only; tools validate their arguments, return a planning verdict like the base commands and never raise; and the optimizer runs no evaluations. Each round ends with a 3–10 line proposal.md.

G Layout structure of RoboDojo seeds

Each evaluation seed is a directory of pre-sampled layouts, 30 to 75 per task. Within a seed, standard layouts differ only in the task objects' positions and yaw (sampled inside per-object rectangles) and in the table and ground materials; random layouts additionally sample 13 to 18 distractor objects from 85 categories with their own poses. A few tasks vary content rather than pose: press by number keeps every object fixed and changes only the two digits; classify objects draws three objects from ten categories. The leaderboard takes the first 50 valid layouts of each seed; our development layouts are the first ten of seed 0.

Table 5. Results on all 42 RoboDojo tasks. Phys. and Dec.: the physical interaction and the decision a task requires (Sec. 5.1): R rigid pick, place or stack; Pu push, slide or press; In tight insertion; Ar articulated object or tool use; Df deformable, fluid or granular; Mv moving objects; G geometry only; S symbols, language or categories; M memory. Official: RoboDojo's official evaluation on a hidden seed, 50 episodes per task (Generalization: 25 standard / 25 cluttered). RGB: our frozen retests on ten new layouts, 10 to 19 of seed 0 (Generalization: standard / cluttered 5 to 9); swap blocks with the benchmark's fix of its button reset and tools developed again under it. Depth net.: frozen retests on the ten seed-0 development layouts (Generalization: five standard / five cluttered). §The optimizer read a note on what the checker requires (Sec. 3.4). Rendered: success in % of the earlier tools with rendered depth; ⋆50 episodes on each of seeds 0–2 (25 per variant for Generalization tasks), otherwise the development layouts; †tools from a run with checker notes, ‡tools rebuilt by a team member (Sec. D). VPP2, PhysicalRSI: leaderboard success in %.

Type	Task	Phys.	Dec.	Official %	RGB	Depth net.	Rendered %	VPP2 %	PhysicalRSI %
Generalization	arrange largest number	R	S	12 / 32	1/5 / 1/5	3/5 / 2/5	80 / 40	4 / 0	1.3 / 21.3
	fold clothes§	Df	G	56 / 24	4/5 / 1/5	2/5 / 1/5	30.7 / 18.7⋆	74.7 / 61.3	41.3 / 34.7
	hang mugs§	In	G	4 / 4	0/5 / 0/5	0/5 / 0/5	40 / 0	64 / 0	0 / 0
	make toast§	Ar	G	0 / 0	0/5 / 0/5	0/5 / 0/5	0 / 0	56 / 22.7	0 / 0
	pack objects into box§	R	S	20 / 12	0/5 / 0/5	0/5 / 0/5	0 / 40	10.7 / 2.7	5.3 / 0
	pour liquid into cup§	Df	G	4 / 0	0/5 / 0/5	1/5 / 2/5	94.7 / 58.7⋆†	68 / 32	93.3 / 9.3
	push T	Pu	G	100 / 92	5/5 / 5/5	2/5 / 2/5	78.7 / 86.7⋆	2.7 / 0	65.3 / 0
	sort nesting dolls by size§	R	S	72 / 32	2/5 / 1/5	1/5 / 0/5	80 / 60†	34.7 / 2.7	16 / 0
	stack blocks	R	G	88 / 88	4/5 / 1/5	4/5 / 2/5	98.7 / 97.3⋆	86.7 / 2.7	9.3 / 0
	stack bowls	R	G	92 / 80	4/5 / 5/5	5/5 / 5/5	90.7 / 94.7⋆	90.7 / 5.3	74.7 / 0
	store laptop and headphones§	Ar	G	0 / 0	0/5 / 0/5	0/5 / 0/5	0 / 0	45.3 / 20	1.3 / 1.3
	sweep blocks§	Ar	G	8 / 8	0/5 / 0/5	0/5 / 0/5	0 / 0	20 / 1.3	0 / 0
Memory	cover blocks	R	M	100	8/10	10/10	100⋆	100	100
	imitate sorting sequence§	R	M	42	3/10	1/10	60†	0	0
	match and pick from conveyor	Mv	M	74	6/10	5/10	56.7⋆	86	7.3
	press by number§	Pu	M	96	10/10	10/10	100⋆†	96	79.3
	swap T	R	M	96	9/10	7/10	82.7⋆	25.3	92.7
	swap blocks§	R	M	78	3/10	0/10	0	26	0
Long-Horizon	classify objects	R	S	78	3/10	3/10	79.3⋆	53.3	40
	fill egg holder§	In	G	0	0/10	0/10	0	0	0
	fill pen holder§	In	G	0	0/10	0/10	0	29.3	3.3
	make kong§	R	S	2	0/10	0/10	11⋆†	78.7	82
	organize table§	Ar	S	0	0/10	0/10	0	7.3	4
	play stacking toy	In	S	18	0/10	0/10	73.3⋆	23.3	0
	play tic tac toe§	R	S	96	10/10	6/10	33.3⋆	9.3	94
	put bottles into dustbin	R	G	58	6/10	2/10	63.3⋆	94.7	75.3
Precision	build tower	In	G	30	2/10	0/10	56⋆‡	72.7	28.7
	deposit coin§	In	G	16	2/10	0/10	29.3⋆†	21.3	56
	fasten screws§	In	G	18	2/10	0/10	0	35.3	0
	insert key§	In	G	0	0/10	0/10	80†	0	39.3
	insert tubes	In	G	22	3/10	2/10	52⋆	74.7	66
	play Xylophone§	Ar	G	58	6/10	0/10	0	0	0.7
	plug in charger§	In	G	32	1/10	0/10	30	16.7	30.7
	pour balls into vase§	Df	G	80	9/10	4/10	82.7⋆†	60.7	38.7
Open	align blocks	Pu	G	78	7/10	9/10	92⋆	0	86
	classify objects by language	R	S	72	3/10	1/10	40	2	6
	general pickup	R	S	82	9/10	8/10	74.7⋆	18	47.3
	pick from conveyor by image§	Mv	S	28	2/10	1/10	10†	0	0
	pour by language§	Df	S	0	1/10	2/10	90†	0	0
	solve equation	R	S	86	8/10	8/10	78⋆	0	60
	stack blocks by language	R	S	96	10/10	7/10	83.3⋆	12.7	0
	store tools in toolbox§	In	S	0	0/10	0/10	0	0	0

Table 6. RGB tools per task. Commands: task commands besides the shared tools; Lines: lines of Python in the enabled tools and the modules they import; Rounds: optimizer rounds of the RGB run; View: executed steps spent on viewing moves in the retest, in %; Retest: successes on the ten development layouts.

Type	Task	Commands	Lines	Rounds	View	Retest
Generalization	arrange largest number	7	1,785	23	0.9	3/10
	fold clothes	7	2,310	21	0.0	6/10
	hang mugs	13	2,944	12	12.4	0/10
	make toast	15	2,022	11	4.5	1/10
	pack objects into box	7	1,758	16	0.2	0/10
	pour liquid into cup	4	2,548	31	20.3	1/10
	push T	4	1,014	13	7.8	10/10
	sort nesting dolls by size	8	3,883	18	0.9	4/10
	stack blocks	6	917	14	0.0	8/10
	stack bowls	2	225	11	0.0	8/10
	store laptop and headphones	5	1,248	18	2.6	0/10
	sweep blocks	14	1,264	12	0.7	0/10
Memory	cover blocks	5	407	12	0.0	10/10
	imitate sorting sequence	22	3,469	17	0.0	8/10
	match and pick from conveyor	7	827	20	1.8	7/10
	press by number	2	924	16	2.1	9/10
	swap T	5	1,204	14	0.0	10/10
	swap blocks	4	1,857	26	0.0	0/10
Long-Horizon	classify objects	7	1,445	24	0.9	4/10
	fill egg holder	6	2,703	24	40.2	0/10
	fill pen holder	12	3,090	13	0.1	0/10
	make kong	16	3,248	14	1.9	0/10
	organize table	14	2,189	17	3.4	0/10
	play stacking toy	6	804	12	2.3	0/10
	play tic tac toe	9	1,518	16	0.0	9/10
	put bottles into dustbin	9	1,753	21	0.0	2/10
Precision	build tower	6	2,257	13	4.9	1/10
	deposit coin	2	2,552	26	16.2	0/10
	fasten screws	4	1,585	14	1.6	0/10
	insert key	7	2,963	41	5.4	0/10
	insert tubes	5	2,630	29	3.1	3/10
	play Xylophone	7	1,058	19	17.0	6/10
	plug in charger	12	2,489	23	15.6	0/10
	pour balls into vase	9	1,607	14	4.7	7/10
Open	align blocks	3	716	13	0.0	8/10
	classify objects by language	9	1,917	25	0.0	4/10
	general pickup	3	260	13	2.0	7/10
	pick from conveyor by image	6	2,564	19	21.9	0/10
	pour by language	15	3,546	19	1.5	0/10
	solve equation	4	764	18	10.6	7/10
	stack blocks by language	5	1,082	13	0.0	9/10
	store tools in toolbox	14	3,088	24	0.4	0/10
All		327	78,434	769	3.6	152/420

Table 7. Run with rendered depth and checker notes: what a note changes. Retests on the ten development layouts (standard / random for Generalization tasks) and success in % over seeds 0–2 (fold clothes, in parentheses: the tools of the first round, which were kept).

Task	Round 1	Round 2	%	What the note supplied
press by number	0/10	10/10, 10/10	100	confirm after *each* group; no extra press
pour liquid into cup	0/5 / 2/5	5/5 / 5/5	94.7 / 58.7	judged once the bottle is upright again
pour balls into vase	2/10	10/10, 9/10	82.7	three end conditions; reach failures
deposit coin	3/10	6/10	29.3	coin must fall through; wrist roll drops it
fold clothes	3/5 / 0/5	2/10, 4/10	(30.7 / 18.7)	keypoint windows and fold order
make kong	0/10	8/10	11.0	a second, unstated stage
insert key	0/10	0/10	–	checker conditions; reach of the turn